

//binary search.

```
#include <iostream>

int main() {

    int arr[] = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};

    int n = sizeof(arr) / sizeof(arr[0]);

    int target = 7;

    int left = 0;

    int right = n - 1;

    int foundIndex = -1;

    while (left <= right) {

        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {

            foundIndex = mid;

            break;

        } else if (arr[mid] < target) {

            left = mid + 1;

        } else {

            right = mid - 1;

        }

    }

    if (foundIndex != -1) {

        std::cout << "Element " << target << " found at index " << foundIndex << std::endl;

    } else {

        std::cout << "Element " << target << " not found in the array." << std::endl;

    }

    return 0;

}
```

2 // dfs code

```
#include <iostream>

#include <vector>

#include <stack>

int main() {

    int numNodes = 6; // Number of nodes in the graph

    std::vector<std::vector<int>> adjList(numNodes);

    // Define the graph's adjacency list
    adjList[0] = {1, 2};
    adjList[1] = {0, 3, 4};
    adjList[2] = {0, 5};
    adjList[3] = {1};
    adjList[4] = {1};
    adjList[5] = {2};

    std::vector<bool> visited(numNodes, false);
    std::stack<int> nodeStack;

    // Starting node for DFS (e.g., starting at node 0)
    int startNode = 0;

    // Push the starting node onto the stack
    nodeStack.push(startNode);

    while (!nodeStack.empty()) {

        int currentNode = nodeStack.top();
        nodeStack.pop();
```

```
    if (!visited[currentNode]) {
        visited[currentNode] = true;
        std::cout << "Visited node: " << currentNode << std::endl;

        // Push adjacent unvisited nodes onto the stack
        for (int neighbor : adjList[currentNode]) {
            if (!visited[neighbor]) {
                nodeStack.push(neighbor);
            }
        }
    }
}

return 0;
}

// bfs

#include <iostream>
#include <vector>
#include <queue>

int main() {
    int numNodes = 6; // Number of nodes in the graph
    std::vector<std::vector<int>>> adjList(numNodes);

    // Define the graph's adjacency list
    adjList[0] = {1, 2};
```

```
adjList[1] = {0, 3, 4};
adjList[2] = {0, 5};
adjList[3] = {1};
adjList[4] = {1};
adjList[5] = {2};

std::vector<bool> visited(numNodes, false);
std::queue<int> nodeQueue;

// Starting node for BFS (e.g., starting at node 0)
int startNode = 0;

// Push the starting node onto the queue
nodeQueue.push(startNode);

while (!nodeQueue.empty()) {
    int currentNode = nodeQueue.front();
    nodeQueue.pop();

    if (!visited[currentNode]) {
        visited[currentNode] = true;
        std::cout << "Visited node: " << currentNode << std::endl;
        // Push unvisited neighbors onto the queue
        for (int neighbor : adjList[currentNode]) {
            if (!visited[neighbor]) {
                nodeQueue.push(neighbor);
            }
        }
    }
}
```

```
    }  
}  
return 0;  
}  
  
// Vector  
  
#include <iostream>  
  
#include <vector>  
  
int main() {  
    // Create an empty vector of integers  
    std::vector<int> numbers;  
  
    // Add elements to the vector  
    numbers.push_back(10);  
    numbers.push_back(20);  
    numbers.push_back(30);  
    numbers.push_back(40);  
  
    // Access elements using [] operator  
    std::cout << "First element: " << numbers[0] << std::endl;  
  
    // Access elements using at() method (with bounds checking)  
    std::cout << "Second element: " << numbers.at(1) << std::endl;  
  
    // Access elements using iterators  
    std::cout << "Elements in the vector: ";  
    for (std::vector<int>::iterator it = numbers.begin(); it != numbers.end(); ++it) {  
        std::cout << *it << " ";  
    }
```

```
}  
  
std::cout << std::endl;  
  
// Size of the vector  
std::cout << "Size of the vector: " << numbers.size() << std::endl;  
  
// Check if the vector is empty  
if (numbers.empty()) {  
    std::cout << "Vector is empty." << std::endl;  
} else {  
    std::cout << "Vector is not empty." << std::endl;  
}  
  
return 0;  
}
```

string

string input - gets() -count before endl

 - getline - char type array - cin.getline (rakib,20)

 -class type array- getline (cin,rakib)

string lenth - char type array - int L=strlen(str)

 - string class - name.size() / name.length()

string reverse - char type array - strrev(str)

string copy - char type array - strcpy (str2,str1) str2= container,str1= donar

string concrate - char type array - strcat(str2,str1) str2=jetar ses e jog korbo,str1=add value

string compare - char type array - strcmp (str1,str2)

 if come pare is same = return 0 or if compare is different = return -1

string replace - string class - string name= "ashraful " // name.replace(0,8,"rakib")

 0 = 1st koto index teke delet hbe , 8= ses e koto index projonto delet hbe

//Binary Search

```
#include <bits/stdc++.h>
using namespace std;
int BinarySearch(int arr[], int target, int low, int high);
int main(){
    int Size, i;
    cout << "Enter Array Size: ";
    cin >> Size;
    int Array[Size];
    for( i = 0; i < Size; i++ ) {
        cout << "Enter Value: ";
        cin >> Array[i];
    }
    sort(Array, Array+Size);
    cout << "After Sorting: ";
    for( i = 0; i < Size; i++ ) {
        cout << Array[i] << " ";
    }
    int t;
    cout << "\nEnter Target: ";
    while( cin >> t ){
        int index;
        index = BinarySearch(Array, t, 0, Size-1);
        if( index != -1 )
            cout << "Located At Index: " << index << endl;
        else
            cout << "Not Found" << endl;

        cout << "\nEnter Target: ";
    }
    return 0;
}

int BinarySearch(int arr[], int target, int low, int high){
    int mid;
    while( low <= high ) {
        mid = low + ( high - low ) / 2;
        if( arr[mid] == target )
            return mid;

        if( arr[mid] < target )
            low = mid + 1;
        else if( arr[mid] > target )
            high = mid - 1;
    }
    return -1;
}
```



```
//String Handling
```

//Stack

```
#include <iostream>
#include <stack>
using namespace std;
int main()
{
    stack<int> num;
    num.push(10);
    cout << num.top() << endl;
    num.pop();
    cout << num.top() << endl;
    cout << num.size() << endl;
    cout << num.empty() << endl;
    return 0;
}
```

//Queue

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    queue<string>str;
    str.push("hi");
    string s;
    cin >> s;
    str.push(s);
    cin >> s;
    str.push(s);
    cout << str.front() << endl;
    cout << str.back() << endl;
    str.pop();
    cout << str.size() << endl;
    cout << str.empty() << endl;
    return 0;
}
```

//Pair

```
#include <bits/stdc++.h>
using namespace std;
int main()
```

```
{
    int sizee;
    cin >> sizee;
    pair<int, string> p1, p2, p3, p[sizee];
    for( int i = 0; i < sizee; i++ )
    {
        cin >> p[i].first >> p[i].second;
    }
    for( int i = 0; i < sizee; i++ )
    {
        cout << p[i].first << " " << p[i].second << endl;
    }
    int target;
    cin >> target;
    p1 = p[target];
    cout << p1.first << " " << p1.second << endl;
    p2 = {3, "imran"};
    p3 = make_pair(4, "siam");
    return 0;
}
```

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    pair<int, string> p[5];

    cin >> p[0].first >> p[0].second;
    cin >> p[1].first >> p[1].second;

    cout << p[0].first << " " << p[0].second << endl;
    cout << p[1].first << " " << p[1].second << endl;

    cout << "After Swapping" << endl;

    swap(p[0], p[1]);

    cout << p[0].first << " " << p[0].second << endl;
    cout << p[1].first << " " << p[1].second << endl;
}
```

///If Number Of Elements Greater Than 10^6 We Can't Do Sorting

// Quick sort

```
#include <bits/stdc++.h>
using namespace std;
void swap(int *a, int *b)
```

```
{
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int array[], int low, int high)
{
    int pivot = array[high];
    int i = (low - 1);
    for (int j = low; j < high; j++)
    {
        if (array[j] <= pivot)
        {
            i++;
            swap(&array[i], &array[j]);
        }
    }
    swap(&array[i + 1], &array[high]);
    return (i + 1);
}

void quickSort(int array[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(array, low, high);
        quickSort(array, low, pi - 1);
        quickSort(array, pi + 1, high);
    }
}

int main()
{
    int sizee;
    cout << "Enter Array Size: ";
    cin >> sizee;
    int arr[sizee];
    for( int i = 0; i < sizee; i++ )
    {
        cout << "Enter Value: ";
        cin >> arr[i];
    }
    quickSort(arr, 0, sizee-1);
    for( int i = 0; i < sizee; i++ )
    {
        cout << arr[i] << " ";
    }
    return 0;
}
```

//Find Maximum Or Minimum Value In An Array

```
cout << "Max: " << *max_element(arr, arr + sizee);  
cout << "Min: " << *min_element(arr, arr + sizee);
```

//Find The Index Of Maximum Or Minimum Value In An Array

```
cout << "Index: " << min_element(arr, arr + sizee) - arr;  
cout << "Index: " << max_element(arr, arr + sizee) - arr;
```

//Sort An Array

```
sort(arr, arr+sizee);
```

//Count Digits In A Number

```
log10(num)+1
```

////////////////////////////////

```
#include <bits/stdc++.h>  
using namespace std;  
#define print_this cout << "C++" << endl;  
int main()  
{  
    print_this;  
    print_this  
    return 0;  
}
```

////////////////////////////////

```
int a = 21;  
char c = 'z';  
string s = "GFG";  
float x = 45.5;  
Can be declared as :  
auto a = 21;  
auto c = 'z';  
auto s = "GFG";  
auto x = 45.5;
```

//Floor And Ceiling

```
#include <bits/stdc++.h>  
using namespace std;  
int main()  
{  
    float a = 4.64;  
    cout << floor(a) << endl; //4
```

```
    cout << ceil(a) << endl; //5
    return 0;
}
```

//String Sorting

```
#include <stdio.h>
int main()
{
    int lines, length;
    printf("How Many Lines?\n");
    scanf("%d", &lines);
    printf("Length Of Each Line?\n");
    scanf("%d", &length);
    char word[lines][length];
    getchar();
    for( int i = 0; i < lines; i++ )
    {
        printf("Input Line %d: ", i+1);
        fgets(word[i], sizeof(word[i]), stdin);
    }
    for( int i = 0; i < lines-1; i++ )
    {
        for( int j = i+1; j < lines; j++ )
        {
            if( word[i][0] > word[j][0] )
            {
                char temp[strlen(word[i])];
                strcpy(temp, word[i]);
                strcpy(word[i], word[j]);
                strcpy(word[j], temp);
            }
        }
    }
    printf("\n<<After Sorting In Lexicographical Order>>\n");
    for( int i = 0; i < lines; i++ )
    {
        puts(word[i]);
    }
    return 0;
}
```

//GCD

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    int n, m;
```

```

    cin >> n >> m;
    cout << "GCD: " << __gcd(n,m) << endl;
    return 0;
}

```

//Merge Sort

```

#include <bits/stdc++.h>
using namespace std;
void merrge(int arr[], int p, int q, int r);
void mergesort(int arr[], int l, int r);
void print(int arr[], int limit);
int main()
{
    int s;
    cout << "Array Size: ";
    cin >> s;
    int a[s];
    for( int i = 0; i < s; i++ )
    {
        cout << "Enter Value: ";
        cin >> a[i];
    }
    cout << "Before Sorting" << endl;
    print(a, s);
    cout << endl;
    mergesort(a, 0, s-1);
    cout << "After Sorting" << endl;
    print(a, s);
    cout << endl;
}

```

```

void merrge(int arr[], int p, int q, int r)
{
    int s1 = q - p + 1;
    int s2 = r - q;
    int L[s1];
    int M[s2];
    for( int i = 0; i < s1; i++ )
    {
        L[i] = arr[ p + i ];
    }
    for( int j = 0; j < s2; j++ )
    {
        M[j] = arr[ q + j + 1 ];
    }
    int i = 0, j = 0, k = p;

```

```

while( i < s1 && j < s2 )
{
    if( L[i] <= M[j] )
    {
        arr[k] = L[i];
        i++;
    }
    else
    {
        arr[k] = M[j];
        j++;
    }
    k++;
}
while( i < s1 )
{
    arr[k] = L[i];
    i++;
    k++;
}
while( j < s2 )
{
    arr[k] = M[j];
    j++;
    k++;
}
}

void mergesort(int arr[], int l, int r)
{
    int m;
    if( l < r )
    {
        m = l + ( r - l ) / 2;
        mergesort(arr, l, m);
        mergesort(arr, m+1, r);
        merrge(arr, l, m, r);
    }
}

void print(int arr[], int limit)
{
    for( int i = 0; i < limit; i++ )
    {
        cout << arr[i] << " ";
    }
}

```

//String Handling

```
#include <bits/stdc++.h>

using namespace std;

int main()
{
    string str;

    //getline(cin, str);

    int len;

    ///String Length
    len = str.length();

    cout << len;

    ///Return 1 If String Is Empty
    cout << str.empty() << endl;

    ///For Clearing Whole String
    str.clear();

    ///Merges A String With Other String
    //Added TO str ....
    string temp_str;
    str.append(temp_str);

    char ch;

    ///Returns First Index Character Of String str
    ch = str.at(0);

    ///Make A Substring Of A String
    //substr(starting index, number of elements)
    string temp;
    temp = str.substr(0, 3);

    ///Convert Character Of String text_string to Uppercase Or Lowercase
    temp = toupper(text_string.at(i));
    temp = tolower(text_string.at(i));

    ///Insert A New_String Anywhere(Any Index) Inside Another String
    str.insert(0, new_string);
```

```
///Search For The Index Position Of A Character Or A Substring Inside A String
int index = str.find(s);

///Erase Portion Of A String
//erase(starting index, number of character to erase);
str.erase(0, 4);

///To Swap String
str.swap(src);

///Returns 1 If Char Is Alphabet
isalpha(str.at(i))

///Returns 1 If Char Is Space
isspace(str.at(i))

///Returns 1 If Char Is A Digit
isdigit(str.at(i))

///Reverse A String
reverse(str.begin(), str.end());

///Replace A Portion Of A String With Another String
//replace(start index, number of char to replace, another string);
str.replace(i,1,s);

///Removes Last Char Of A String
str.pop_back();

///Add A Char(Not String) In The End Of A String
str.push_back(ch);

return 0;
}
```

//Vector

```
#include <bits/stdc++.h>

using namespace std;

int main()
{
    vector<int> num;
```



```
int n;

cin >> n;

num.push_back(n);

///Print First Index Of Vector num
cout << num[0] << endl;

///Size Of Vector num
cout << num.size() << endl;

///Make Vector Function And Pass A Vector To A Function
//Function
void print_vector( vector<int> v )
{
    for( int i = 0; i < v.size(); i++ )
    {
        cout << v[i] << endl;
    }
}
//Call Function
print_vector(num)

return 0;
}
```

//Map

```
// create a map with integer keys and string values
map<int, string> student = {{1,"Jacqueline"}, {2,"Blake"}, {3,"Denise"}};

insert() adds an element (key-value pair) to the map
///
erase() removes an element or range of elements from the map

// use itr iterator to point to the first map element
itr = student.begin();

// remove the first element
student.erase(itr);
```

```
// remove the element having key 4
student.erase(4);

// display final map values
cout << "\nFinal Values:" << endl;

for(itr = student.begin(); itr != student.end(); ++itr)
{
    cout << itr->second << " , ";
}

// remove a range of elements
student.erase(student.find(2),student.find(5));
///
clear()   removes all the elements from the map
map<int, string> student;
student[1] = "Jacqueline";
student[2] = "Blake";

cout << student.size(); // Output: 2

student.clear();

cout << student.size(); // Output: 0

///
find()   searches the map for the given key
student[1] = "Jacqueline";
student[2] = "Blake";

// find the key 2 in the student map
// store the return value in iter
iter = student.find(2);

///
size()   returns the number of elements in the map
empty()  returns true if the map is empty

// create a map with integer keys and string values
std::map<int, string> student;

// add element with key 1 and value "Jacqueline"
student[1] = "Jacqueline";

// add element with key 2 and value "Blake"
student[2] = "Blake";

map<int, string> student;
```

```
// use [] operator to add elements
student[1] = "Jacqueline";
student[2] = "Blake";

// use insert() method to add elements
student.insert(make_pair(3, "Denise"));
student.insert(make_pair(4, "Blake"));

// add elements with duplicate keys
student[5] = "Timothy";
student[5] = "Aaron";

for (int i = 1; i <= student.size(); ++i)
{
    cout << "Student[" << i << "]: " << student[i] << endl;
}

///Display Map Values
map<int, string> student;

student[1] = "Jacqueline";
student[2] = "Blake";
student[3] = "Denise";
student[4] = "Aaron";

// declare map iterator
map<int, string>::iterator iter;

// use iterator to display map elements
for (iter = student.begin(); iter != student.end(); ++iter)
{
    cout << iter->first << " - " << iter->second << endl;
}
```